Data Structures and Algorithms LAB – Spring 2026

(BS-IT-F24 Morning & Afternoon)

Lab # 6

Instructions:

- You must complete all tasks individually. Absolutely NO collaboration is allowed. Any traces of plagiarism/cheating or usage of AI tools will result in an **"F"** grade in this course and lab.
- Attempt the following tasks exactly **in the given order**.
- **Indent** your code properly.
- Use meaningful variable and function names. Use the **camelCase** notation.
- Use meaningful prompt lines/labels for all input/output.
- Make sure that there are **NO** ***dangling pointers*** or ***memory leaks*** in your programs.

Task # 1

As discussed in the lecture, the Queue data structure follows the First-In-First-Out (FIFO) principle. Elements in a queue are inserted at the **back** end and removed from the **front** end of the queue. A **Double Ended Queue** (Deque) (pronounced *"deck"*) is a queue in which you can insert and remove elements **both at the *front* and at the *back* of the queue**. In this task, you are required to implement a class **`Deque`** (which will be used to store integers). The **`Deque`** class should have the following public member functions:

- **`Deque (int n)`** Constructor to create a Deque which can contain up to **`n`** integers.
- **`~Deque ()`** Destructor to destroy the Deque.
- **`bool isEmpty ()`** returns true if the Deque is empty, and false otherwise.
- **`bool isFull ()`** returns true if the Deque is full, and false otherwise.
- **`void display ()`** to display the elements of Deque **from front to back**.
- **`void makeEmpty ()`** to make the Deque empty.
- **`bool insertAtFront (int val)`** to insert **`val`** at the front of the queue.
- **`bool insertAtBack (int val)`** to insert **`val`** at the back of the queue.
- **`bool removeFromFront (int& val)`** to remove an integer from the front of the queue and store it in the variable **`val`**.
- **`bool removeFromBack (int& val)`** to remove an integer from the back of the queue and store it in the variable **`val`**.

Note that:

- The **time complexity** of **all member functions** (except the **`display()`** function) of the **`Deque`** class should be **constant i.e. $O(1)$**.
- Your **`Deque`** class should refuse the insertion of a new element ONLY when there are no empty slots left in the **`Deque`**. So, make sure that all the boundary cases are properly handled.

You are also required to write a menu-driven program which allows the user to use all of the aforementioned functionalities of the **Deque** class. A sample menu is shown below (Text shown in **Red** is entered by the user):

```
Enter the size of Deque: 10

1. Insert value at Front
2. Insert value at Back
3. Remove value from Front
4. Remove value from Back
5. Display the Deque
6. Make the Deque Empty
7. Exit

Enter your choice:
```

Task # 2.1

In this task you are going to write a program to implement the algorithm (that we have seen in class) for evaluating **postfix expressions** which involves the four basic arithmetic operators: **+**, **–**, *****, and **/**, and the unary minus operator (**~** will be used to indicate the unary minus operator, in order to distinguish it from the subtraction operator).

Since, this algorithm uses a stack; therefore, firstly you will have to implement a **Stack** class for storing **double** values (during lecture we have already implemented a stack of integers).

Once, you have implemented the **Stack** class, you need to get the **postfix expression** from the user and store it in a null-terminated **char** array (c-string). After that you would implement the following function, which takes the postfix expression present in the null-terminated **char** array (**input**) and evaluates it using the algorithm that we have discussed in the lecture:

```
void evaluatePostfixExpression ( char* input )
```

To trace the actions of the algorithm, this function should also display each token as it is encountered, and the actions (stack operations) that it is performing. For example, the output of your program should resemble the following (text shown in **Red** is entered by the user):

```
Enter the postfix expression: 12 3 ~ 1 - / 4.6 ~ *

Token = 12        Push 12
Token = 3         Push 3
Token = ~         Pop 3       Push -3
Token = 1         Push 1
Token = -         Pop 1       Pop -3      Push -4
Token = /         Pop -4      Pop 12      Push -3
Token = 4.6       Push 4.6
Token = ~         Pop 4.6     Push -4.6
Token = *         Pop -4.6    Pop -3      Push 13.8
End of input

Answer = 13.8
```

Note that in the above example, input provided by the user actually corresponds to the following infix expression: **12 / (-3 – 1) * -4.6**

Note the following important points regarding the input/output format:

- Your program should strictly follow the input/output format shown in the example above.
- As you can see from the above example, the numbers in the input can have more than one digit, and there might also be some floating-point values in the input.
- You can assume that there will be **a space** after each operand or operator given in the input (except the last operator).
- You can assume that there are at most 200 characters (including spaces) in the input string entered by the user.
- You can also assume that the expression entered by the user is valid.

Hint: You might find following library functions helpful in your implementation:

- **`strtok`** in the header file **`<cstring>`**
- **`atof`** in the header file **`<cstdlib>`**
- **`isdigit`** in the header file **`<cctype>`**

Task # 2.2

Modify your program (from **Task # 2.1**) to handle the following cases in which the input is invalid. Your program should display an appropriate error message when one of the following conditions arises:

- Too few operands. For example: **`1 3 + *`**
- Too few operators. For example: **`3 4 5 +`**
- Invalid operator. For example: **`4 6 + 8 &`**

Task # 2.3

Now, modify your implementation to take input from a **text file** in which several postfix expressions are given (one on each line). Your program should evaluate these expressions, and put these expressions and their resulting values in an **output text file**. If some expressions are invalid, your program should store an appropriate error message with that expression.

VERY IMPORTANT

In the next Lab, you may need some or all of the functions from Today's Lab. So, make sure that you have the working implementation of **ALL** the functions of Today's Lab, when you come to the next Lab.